Lecture 1: Introduction to mobile development and architecture

Overview and motivation

- **Definition:** A mobile app is an installable software package (e.g., APK/IPA) that runs directly on a phone, giving full access to device capabilities like GPS, camera, notifications, sensors, and offline storage.
- **Why apps (not just websites):** Better performance, offline-first experiences, access to hardware features (e.g., AR overlays with camera+GPS), and tailored UX that matches OS guidelines.

What is Mobile App Development?

It is the process of creating software applications that run on mobile devices such as smartphones and tablets. These apps can be native, hybrid, or cross-platform, depending on the development approach and technology stack.
The goal is to create responsive, efficient, and user-friendly apps that work smoothly across different operating systems (mainly Android and iOS).

2. Mobile Application Development Approaches

There are three major approaches to building mobile apps, each with its own advantages, disadvantages, and use cases.

A. Native App Development

Definition:
Apps developed specifically for one platform (Android or iOS) using platform-specific languages and tools.

Android: Java or Kotlin using Android Studio

iOS: Swift or Objective-C using Xcode

Example: WhatsApp, Instagram (initially native apps for both OS).

Advantages:

Best Performance: Direct access to device hardware.
Access to Native Features: Camera, GPS, Sensors, etc.
Optimized User Experience: Matches OS look & feel.
Disadvantages:

Two separate codebases → higher cost & effort.
Requires expertise in both Android and iOS.
Longer development time.
Use Case: Apps requiring high performance and deep integration with device hardware — like games or social media apps.

B. Hybrid App Development

Definition:
Hybrid apps are built using web technologies (HTML, CSS, JavaScript) and wrapped inside a WebView, allowing them to run like native apps on multiple platforms.
Frameworks: Ionic, Apache Cordova, PhoneGap
Examples: Early versions of Facebook and Instagram.

Advantages:

Single codebase → runs on both platforms.
Quick and cheaper to develop.
Easier maintenance and updates.

Disadvantages:

Slower performance (runs inside a WebView).
Limited access to some native APIs.
UI/UX may not feel truly "native".

Use Case: Apps with simple interfaces and minimal hardware dependency.

C. Cross-Platform App Development

Definition:
A modern approach where you write one codebase that compiles into native code for both Android and iOS.

Frameworks:

React Native (Meta/Facebook)
Flutter (Google)
Xamarin (Microsoft)

Examples:
Facebook, Uber Eats, Tesla (React Native)
Google Ads, Alibaba (Flutter)

Advantages:

One codebase for Android & iOS.
Close-to-native performance.
Easier maintenance and faster updates.
Large community support.

Disadvantages:

Slightly less optimized than true native.
Some native modules may need custom coding.
Complex native integrations can be tricky.

Use Case: Ideal for startups and medium-scale projects where time, cost, and performance need balance.

Mobile app vs mobile website

Attribute	**Mobile app**	**Mobile website**
Installable	Yes	No
Device features	Full access (GPS/camera/sensors)	Limited
Performance	High, native threads	Depends on browser
Offline	Supported	Limited
Updates	Through stores	Deployed instantly

Lecture 2: Development approaches and React Native basics

Android architecture

- **Application layer:** All apps (native/third-party) built using the same API libraries.
- **Application framework:** Java/Kotlin classes and managers for UI, resources, and hardware abstraction.
- **Android runtime & libraries:** Dalvik/ART run DEX bytecode with libraries like SQLite, SSL, WebKit, media, graphics; Linux kernel provides threading/memory management.
- **Key terms:**
    - **DEX:** Dalvik Executable — compiled bytecode mobile devices can run.
    - **DVM/ART:** Virtual machine/runtime that executes DEX and abstracts hardware.
    - **APK:** Android application package (compiled code + resources).

iOS architecture

- **Core OS:** Low-level features and security (Bluetooth, Security Services, Local Authentication).
- **Core Services:** GPS, telephony, SMS.
- **Media:** Graphics/audio/video libraries.
- **Cocoa Touch:** UI layer (views, touch, controllers).

Android vs iOS differences

Attribute	Android	iOS
OS family	Linux	Unix
IDE	Android Studio	Xcode
Langs	Java/Kotlin	Swift/Objective-C
Package	.apk	.ipa
Ecosystem	Open source	Closed system

3. Introduction to React Native

React Native is an open-source framework by Meta (Facebook) for building cross-platform apps using JavaScript and React.

It allows developers to use the same codebase for Android and iOS and still produce native UI components (not web views).

The React Native "Bridge"

The bridge is what connects JavaScript code with native modules.

Flow:

The app logic is written in JavaScript.
React Native translates it into native UI instructions.
The native OS (Android/iOS) renders the UI using real native components.

React (Web) Flow:
JavaScript (React Code)
↓
React Virtual DOM
↓
Browser DOM

↓

HTML + CSS rendered

React Native Flow:

JavaScript (React Native Code)
↓
React Native Bridge
↓
Native UI Components
↓
Rendered by Mobile OS

This bridge architecture ensures that your app performs like a real native application, not a web page.

4. React Native Components

Components are the building blocks of every React Native application.
Each component defines how a part of your app looks (UI) and how it behaves (logic).

Example:

```
import React from 'react';
import { View, Text, Button } from 'react-native';

export default function App() {
  return (
    <View>
      <Text>Hello React Native!</Text>
      <Button title="Click Me" onPress={() => alert('Button Pressed!')} />
    </View>
  );
}
```

Explanation:

<View> → Like a in HTML, used as a layout container.
<Text> → Displays text.
<Button> → Creates a clickable button.

Types of Components

Core Components:

Built-in elements provided by React Native (e.g. <View>, <Text>, <Button>).

Custom Components:

Components you create yourself by combining core ones.

Example:

```
function Greeting(props) {
```

```
  return <Text>Hello, {props.name}!</Text>;
}
```

Third-party Components:

Ready-made UI components from external libraries.

Examples: React Native Paper, Native Base, React Native Elements.

5. Component Hierarchy

React Native apps are structured as a tree of nested components.

Example Hierarchy:

App

```
├── ProfileScreen
│   ├── Header
│   ├── ProfileImage
│   ├── UserName
│   ├── StatsSection
│   │   ├── Followers
│   │   ├── Following
│   │   └── Posts
│   └── ActionButtons
│       ├── FollowButton
│       └── MessageButton
```

Each child component handles a specific responsibility, making the app modular and easy to maintain.

Package management

- **NPM:** Default package manager with Node.js; commands: npm install, npm start.
- **Yarn:** Faster, reliable for large projects; commands: yarn add, yarn start.
- **package.json:** Declares dependencies (e.g., react, react-native).

Approaches overview

Approach	Stack	Strengths	Trade-offs
Native	Android Studio (Java/Kotlin), Xcode (Swift/Obj-C)	Best performance, full features, perfect UX	Two codebases, higher cost/time
Hybrid	Ionic, Cordova/PhoneGap	One codebase, faster/cheaper	WebView-dependent performance, limited native feel
Cross-platform	React Native, Flutter, Xamarin	One codebase, near-native performance, native components	Edge cases need platform code; complex native integrations can be tricky

7. Summary

Concept	Description	Example Tools
Native Development	**Separate code for each platform**	**Android Studio, Xcode**
Hybrid Development	**Web code inside native shell**	**Ionic, Cordova**
Cross-Platform Development	**One JS code for both**	**React Native, Flutter**
Component-Based Architecture	**UI built from reusable blocks**	**<View>, <Text>, <Button>**
Package Management	**Install and manage libraries**	**NPM, Yarn**

React (Web) vs React Native (Mobile)

Feature	React (Web)	React Native
Output	**HTML elements (, <p>)**	**Native components (<View>, <Text>)**
Rendered by	**Browser (DOM)**	**Mobile OS (via native bridge)**
Platform	**Web browsers**	**Android & iOS**
Example	** Hello Web! **	**<View><Text>Hello Mobile!</Text></View>**

Lecture 3: Props, state, and hooks

Props

What are Props?

Props, short for properties, are like inputs for a React component. Think of a component as a small machine. Props are the knobs, buttons, or settings you give to that machine to tell it how it should behave or look.

In simpler words, props allow you to send data from one component (parent) to another component (child). For example, if you have a button component, props can tell it what text to show or what color it should be.

js

```
function Greeting({ name }) {
  return <Text>Hello, {name}!</Text>;
}
export default function App() {
  return (
    <View>
      <Greeting name="Ahmed" />
      <Greeting name="Ali" />
    </View>
  );
}
```

Key Points About Props

• Read-only: Props cannot be changed by the child component. The child can only use them. The parent decides the values.

• Used for passing data: Props are the way to pass information from parent components to child components.

Advantages of Props

1. Reusability → You can use the same component multiple times with different data. Example: One Button component can show "Submit", "Cancel", or "Next" depending on the prop.

2. Flexibility → Components can adapt to different inputs. Example: A Card component can show different titles or images depending on the props it receives.

3. Separation of Concerns → The parent component handles logic, like fetching data, while the child component handles display, like showing a card or button.

Supported types: String, number, boolean, arrays, objects, functions (callbacks).

1. Callback Props Definition:

• When a parent component passes a function as a prop to the child, the child can send data back or trigger an action in the parent.

• Useful when child needs to communicate with parent.

Key Point: Callback props allow child → parent communication.

State:

Definition: Local, mutable data inside a component; changes trigger re-render.

State is a built-in object that stores data inside a component that can change over time.
• When state changes, React automatically re-renders the component to show the update.
• State is private to the component; other components cannot access it.
Example:

js

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <View>
      <Text>Count: {count}</Text>
      <Button title="Increase" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```

Props vs state

Feature	**Props**	**State**
Owner	Parent	Component
Mutable	No	Yes
Re-render	Yes	Yes
Analogy	Function parameter	Local variable

Hooks overview:

Hooks are built-in functions in React that allow functional components to use state, side effects, context, and more, without using classes.

- **useState:** Manage local state.
- **useEffect:** Side effects (API calls, timers, subscriptions); with cleanup.

- **useContext:** Share data without prop drilling.
- **useRef:** Store mutable values; reference elements.
- **useMemo/useCallback:** Performance optimizations.
- **useReducer:** Structured state management.

useEffect patterns

js

```
// Runs on every render (rarely needed)
useEffect(() => { /* side effect */ });
// Runs only once (mount)
useEffect(() => { /* init */ return () => {/* unmount cleanup */} }, []);
// Runs when dependencies change
useEffect(() => { /* respond to change */ }, [depA, depB]);
```

Arrow Functions

• Shorter and simpler way to write functions in JavaScript.

Example

Conventional vs Arrow:

```
// Conventional
function sayHello() {
 console.log("Hello!"); }

// Arrow Function
 const sayHello = () => {
console.log("Hello!"); }
// With return value
 const add = (a, b) => a + b;
```

React Hooks Example – Timer

```
import React, { useState, useEffect } from 'react';
import { Text, View } from 'react-native';

 export default function Timer() {
const [seconds, setSeconds] = useState(0);

useEffect(() => {
const timer = setInterval(() => setSeconds(s => s + 1), 1000);
return () => clearInterval(timer); // cleanup on unmount
 }, []);

 Return(
 <View>
```

<Text> Time: {seconds}s </Text>
</View>
); }

Explanation:
1. useState(0) → Keeps track of seconds.
2. useEffect() → Starts a timer when component loads.
3. setInterval → Increases seconds every 1 second.
4. return () => clearInterval(timer) → Cleans up the timer when component unmounts.

Output:
Time: 0s
Time: 1s
Time: 2s
Time: 3s

Key Points to Remember
• Props: Data from parent → child, read-only, reusable.
• Callback Props: Child can trigger parent function.
• State: Component's private, dynamic data.
• useState: For storing state.
• useEffect: For side effects like timers or API calls.
• Custom Hooks: Reuse logic across components.
• Arrow Functions: Shorter function syntax.

Lecture 4: Stylesheets and Flexbox

StyleSheet API

- **StyleSheet.create:** Define named, pre-processed style objects.
- **StyleSheet.flatten:** Merge style arrays into a single object.
- **StyleSheet.hairlineWidth:** Thinnest border width.
- **StyleSheet.absoluteFillObject:** Position to fill entire parent.

js

```
const styles = StyleSheet.create({
  container: { backgroundColor: 'black', padding: 20, alignItems: 'center' },
  title: { fontSize: 20, color: 'green' }
});
```

Flexbox essentials

- **Axis:** Main axis depends on flexDirection (row or column).
- **Spacing:** justifyContent (main axis), alignItems (cross axis).
- **Sizing:** flex (relative space share), flexWrap (wrap lines), alignSelf overrides cross-axis alignment.

Examples

js

```
// Row with spaced boxes
<View style={{ flexDirection: 'row', justifyContent: 'space-around', alignItems: 'center' }}>
  <View style={{ width: 50, height: 50, backgroundColor: 'red' }} />
  <View style={{ width: 50, height: 50, backgroundColor: 'green' }} />
  <View style={{ width: 50, height: 50, backgroundColor: 'blue' }} />
</View>

// Vertical spacing and center
<View style={{ flexDirection: 'column', justifyContent: 'space-between', alignItems: 'center', height:
300 }}>
  <View style={{ width: 60, height: 60, backgroundColor: 'orange' }} />
  <View style={{ width: 60, height: 60, backgroundColor: 'purple' }} />
  <View style={{ width: 60, height: 60, backgroundColor: 'pink' }} />
</View>


// Proportional flex
<View style={{ flex: 1 }}>
  <View style={{ flex: 1, backgroundColor: 'skyblue' }} />
  <View style={{ flex: 2, backgroundColor: 'lightgreen' }} />
</View>
```

Lecture 5: Component life cycle

Class components

- **Pattern:** this.state + this.setState, lifecycle methods — powerful but verbose.
- **Key lifecycle methods:**
    - **Mounting:** constructor → render → componentDidMount
    - **Updating:** shouldComponentUpdate → render → componentDidUpdate
    - **Unmounting:** componentWillUnmount
- **Example:**

js

```
import React, { Component } from 'react';
import { View, Text, Button } from 'react-native';

export default class Counter extends Component {
  state = { count: 0 };
  componentDidMount() { console.log('Mounted'); }
  componentDidUpdate(prevProps, prevState) { console.log('Updated'); }
  componentWillUnmount() { console.log('Unmounted'); }
```

```
 render() {
  return (
   <View>
    <Text>Count: {this.state.count}</Text>
    <Button title="Increase" onPress={() => this.setState({ count: this.state.count + 1 })} />
   </View>
  );
 }
}
```

Functional components with hooks

- **Modern approach:** cleaner, no this, logic via hooks.
- **Lifecycle equivalents with** useEffect**:**
  - **componentDidMount:** useEffect(..., [])
  - **componentDidUpdate:** useEffect(..., [deps])
  - **componentWillUnmount:** return a cleanup function from useEffect.

js

```
import React, { useState, useEffect } from 'react';
import { View, Text, Button } from 'react-native';

export default function Counter() {
 const [count, setCount] = useState(0);
 // Mount and update
 useEffect(() => {
  console.log('Mounted or updated');
 });

 // Mount only, with cleanup
 useEffect(() => {
  const timer = setInterval(() => setCount(c => c + 1), 1000);
  return () => clearInterval(timer); // cleanup (unmount)
 }, []);

 return (
  <View>
   <Text>Count: {count}</Text>
   <Button title="Add" onPress={() => setCount(count + 1)} />
  </View>
 );}
```

Common lifecycle patterns and pitfalls

- **Avoid side effects inside render:** use useEffect or lifecycle methods.
- **Cleanup everything you start:** timers, subscriptions, event listeners.
- **Dependency arrays matter:** wrong dependencies cause stale values or infinite loops. Prefer functional state updates (setCount(c => c + 1)) to avoid stale closures.
- **Batching updates:** React batches multiple setState/setCount calls; rely on functional updates when the next state depends on previous state.

Lecture 6: Navigation patterns in React Native

Core concepts

- **Navigation container:** Root state manager for navigation, required for any setup.
- **Navigators:** Organize and present screens with different UX patterns:
    - **Stack:** Push/pop — ideal for flows (Home → Details → Checkout).
    - **Tabs:** Parallel screens with quick switching (Home, Settings).
    - **Drawer:** Side menu that slides in (Home, Profile, About).

```
js
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator();
export default function App() {
 return (
  <NavigationContainer>
   <Stack.Navigator>
    <Stack.Screen name="Home" component={HomeScreen}/>
    <Stack.Screen name="Details" component={DetailsScreen}/>
   </Stack.Navigator>
  </NavigationContainer>
 );
}
```

Passing data between screens

- **Send params:**

```
js

navigation.navigate('Details', { user: 'Sarah', age: 21 });
```

- **Receive params:**

```
js

import { useRoute } from '@react-navigation/native';
const { params } = useRoute();
<Text>{params.user}</Text>
```

Custom headers and screen options

```
js

<Stack.Screen
 name="Home"
 component={HomeScreen}
 options={{
```

```
   title: 'Dashboard',
   headerStyle: { backgroundColor: '#4CAF50' },
   headerTintColor: '#fff',
   headerRight: () => <Button title="Info" onPress={() => alert('Toolbar Button Pressed!')} />
  }}
/>
```

Nested navigation

- **Compose navigators (e.g., Drawer → Tabs → Stack):**

js

```
<Drawer.Navigator>
  <Drawer.Screen name="HomeTabs" component={Tabs} />
  <Drawer.Screen name="ProfileStack" component={ProfileStack} />
</Drawer.Navigator>
```

Hooks and TypeScript

- **Hooks:** useNavigation() for actions; useRoute() for params.
- **Type safety:** Define RootStackParamList and use it with createNativeStackNavigator to catch param errors early.

ts

```
// types.ts
export type RootStackParamList = {
  Home: undefined;
  Details: { name: string; age: number };
};
```

Common navigation pitfalls

- **Forgetting NavigationContainer:** screens render but navigation state isn't managed.
- **Param name mismatches:** itemId vs itemid → undefined.
- **Wrong screen names:** navigate('Profile') must match the registered screen name.
- **Deep link config:** needed to map URLs like myapp://profile/123 to screen routes.

Lecture 7: Working with APIs, async/await, and FlatList

APIs and HTTP basics

- **API:** A contract (URLs + methods + data formats) for requesting/receiving data/services.
- **Common methods:** GET (fetch data), POST (create), PUT/PATCH (update), DELETE (remove).
- **Response handling:** Status codes (200 OK, 201 Created, 4xx client errors, 5xx server errors), JSON payloads.

fetch vs Axios

Aspect	fetch	Axios
Availability	Built-in	Library
JSON parsing	await res.json()	res.data already parsed
Interceptors	Manual	Built-in request/response interceptors
Errors	Only network errors throw; handle res.ok manually	Throws on non-2xx; simpler error handling
TypeScript	Manual types	Generics for typed responses

Async/Await pattern

js

```
const fetchUsers = async () => {
  setLoading(true);
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/users');
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const data = await response.json();
    setUsers(data);
  } catch (e) {
    console.error('Error fetching users:', e);
  } finally {
    setLoading(false);
  }
};

useEffect(() => { fetchUsers(); }, []);
```

Axios pattern

js

```
import axios from 'axios';

useEffect(() => {
  const fetchUsers = async () => {
    setLoading(true);
    try {
      const res = await axios.get('https://jsonplaceholder.typicode.com/users');
      setUsers(res.data);
    } catch (e) {
      console.error('Error fetching users:', e);
```

```
  } finally {
    setLoading(false);
  }
 };
 fetchUsers();
}, []);
```

FlatList essentials

- **Why FlatList:** Efficiently renders long lists using virtualization (only visible items are mounted).
- **Required props:** data, renderItem, keyExtractor.
- **Useful additions:** ItemSeparatorComponent, ListHeaderComponent, ListFooterComponent, onEndReached (pagination), initialNumToRender.

js

```
<FlatList
 data={users}
 keyExtractor={(item) => item.id.toString()}
 renderItem={({ item }) => (
  <Text style={{ padding: 10, fontSize: 16 }}>
   {item.name} ({item.email})
  </Text>
 )}
/>
```

API/FlatList pitfalls

- **Missing keys:** Poor performance; always use stable keyExtractor.
- **SetState after unmount:** Causes memory leaks; cancel requests/cleanup effects.
- **Infinite loops:** Don’t update state inside useEffect without a proper dependency array.
- **Error paths:** Always handle catch and display user-friendly messages/spinners.

PRACTISE QUESTION FOR OUTPUT/ LAYOUT:

Predict the Output:

CODE: 1

PROP E.G

```
function Greeting({ name }) {
  return <Text>Hello, {name}!</Text>;
}
export default function App() {
  return (
    <View>
      <Greeting name="Ali" />
      <Greeting name="Sara" />
    </View>
  );
}
```

CODE 2:

State Update

```
export default function Counter() {
  const [count, setCount] = useState(0);
  useEffect(() => setCount(5), []);
  return <Text>Count: {count}</Text>;
}
```

Code: 3 (Arrow Function)

```
const multiply = (a, b) => a * b;
<Text>{multiply(3, 4)}</Text>
```

Code: 4 (useEffect Timer)

```
js
export default function Timer() {
  const [seconds, setSeconds] = useState(0);
  useEffect(() => {
    const timer = setInterval(() => setSeconds(s => s + 1), 1000);
    return () => clearInterval(timer);
  }, []);
  return <Text>Time: {seconds}s</Text>;
}
```

Code: 5. (Object Prop)

```
const user = { name: "Sara", age: 22 };
<Text>{user.name}</Text>
```

Find the Errors:

Q1. Missing Import

```
js
import { View } from 'react-native';
export default function App() {
  return (
    <View>
      <Text>Hello World</Text>
    </View>
  );
}
```

Q2. Wrong Prop Name

```
js
<Greeting username="Ali" />
```

Q3. TypeScript Error

```
ts
type Item = {
  itemName: string;
  itemPrice; // missing type
}
```

Q4. Invalid Button

```
js
<Button title="Click" onPress={alert("Clicked")} />
```

Q5. Style Error

```
js
const styles = StyleSheet.create({
  container: {
    margin Vertical: 10, //
  }
});
```

Layout Drawing:

Q1. Stack Navigation

```
js
<NavigationContainer>
  <Stack.Navigator>
    <Stack.Screen name="Home" component={HomeScreen} />
    <Stack.Screen name="Details" component={DetailsScreen} />
  </Stack.Navigator>
</NavigationContainer>
```

Q2. Drawer Navigation

```
js
<Drawer.Navigator>
  <Drawer.Screen name="History" component={History} />
  <Drawer.Screen name="Support" component={Support} />
  <Drawer.Screen name="Setting" component={Setting} />
</Drawer.Navigator>
```

Q3. FlatList

```
js
<FlatList
  data={[{name: "Ali"}, {name: "Sara"}]}
  renderItem={({ item }) => <Text>{item.name}</Text>}
/>
```

Q4. Flexbox Row

```
js
<View style={{ flexDirection: 'row', justifyContent: 'space-around' }}>
  <View style={{ width: 50, height: 50, backgroundColor: 'red' }} />
  <View style={{ width: 50, height: 50, backgroundColor: 'blue' }} />
</View>
```

Q5. Profile Screen

```
js
function ProfileScreen({ route }) {
  return <Text>User: {route.params.user}</Text>;
}
```

If user = "Ali", draw the UI.